Unit 3

- Classification And Prediction: Issues Regarding Classification and Prediction
- Classification by Decision Tree Induction
- Bayesian Classification
- Classification by Back propagation
- Support Vector Machines
- Other Classification Methods
- Prediction
- Classifier Accuracy.

What Is Classification? What Is Prediction?

A bank loans officer needs analysis of her data in order to learn which loan applicants are “safe” and which are “risky” for the bank.

A marketing manager at AllElectronics needs data analysis to help guess whether a customer with a given profile will buy a new computer.

A medical researcher wants to analyze breast cancer data in order to predict which one of three specific treatments a patient should receive.

In each of these examples, the data analysis task is classification, where a model or classifier is constructed to predict categorical labels, such as “safe” or “risky” for the loan application data; “yes” or “no” for the marketing data; or “treatment A,” “treatment B,” or “treatment C” for the medical data.

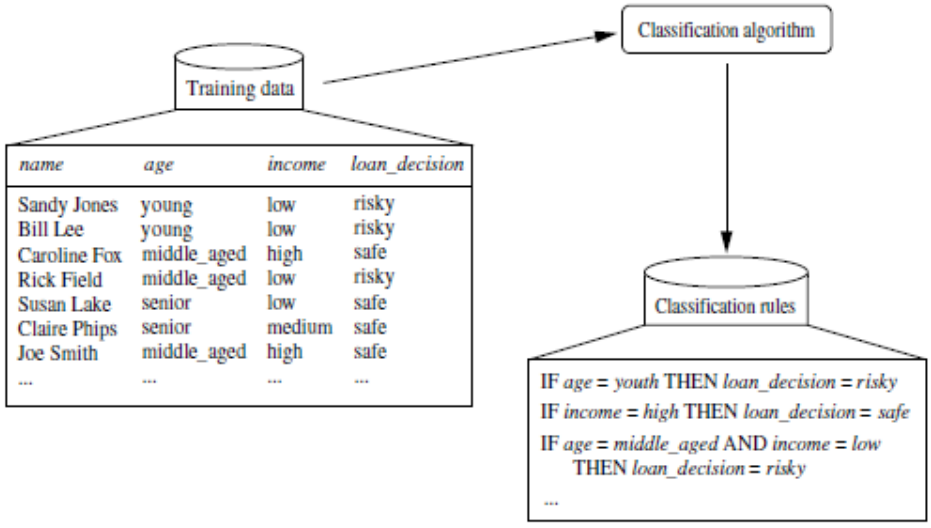
These categories can be represented by discrete values, where the ordering among values has no meaning. For example, the values 1, 2, and 3 may be used to represent treatments A, B, and C, where there is no ordering implied among this group of treatment regimes.

Suppose that the marketing manager would like to predict how much a given customer will spend during a sale at AllElectronics. This data analysis task is an example of numeric prediction, where the model constructed predicts a continuous-valued function.

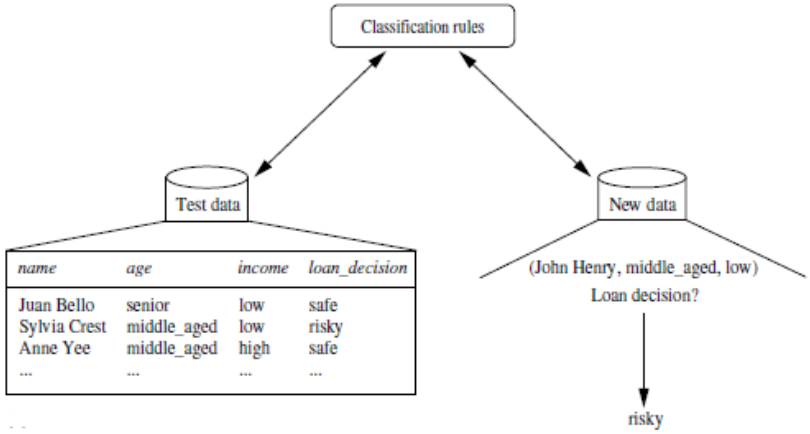
Data classification is a two-step process

In the first step,

a classifier is built describing a predetermined set of data classes or concepts. This is the learning step (or training phase), where a classification algorithm builds the classifier by analyzing or "learning from" a training set made up of database tuples and their associated class labels.



The data classification process: (a) Learning: Training data are analyzed by a classification algorithm. Here, the class label attribute is loan decision, and the learned model or classifier is represented in the form of classification rules



(b) Classification: Test data are used to estimate the accuracy of the classification rules. If the accuracy is considered acceptable, the rules can be applied to the classification of new data tuples.

Data prediction is a two-step process, similar to that of data classification. However,

for prediction, we lose the terminology of "class label attribute" because the attribute for which values are being predicted is continuous-valued (ordered) rather than categorical(discrete-valued and unordered). The attribute can be referred to simply as the predicted attribute.

Issues regarding classification and prediction:

There are two issues regarding classification and prediction they are Issues

(1): Data Preparation Issues

(2): Evaluating Classification Methods

1. Preparing the Data for Classification and Prediction

The following preprocessing steps may be applied to the data to help improve the accuracy, efficiency, and scalability of the classification or prediction process.

Data cleaning:

This refers to the preprocessing of data in order to remove or reduce noise (by applying smoothing techniques, for example) and the treatment of missing values (e.g., by replacing a missing value with the most commonly occurring value for that attribute, or with the most probable value based on statistics). Although most classification algorithms have some mechanisms for handling noisy or missing data, this step can help reduce confusion during learning

Relevance analysis:

Many of the attributes in the data may be redundant. Correlation analysis can be used to identify whether any two given attributes are statistically related. For example, a strong correlation between attributes A1 and A2 would suggest that one of the two could be removed from further analysis.

A database may also contain irrelevant attributes. Attribute subset selection can be used in these cases to find a reduced set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes.

Hence, relevance analysis, in the form of correlation analysis and attribute subset selection, can be used to detect attributes that do not contribute to the classification or prediction task.

Ideally, the time spent on relevance analysis, when added to the time spent on learning from the resulting "reduced" attribute (or feature) subset, should be less than the time that would have been spent on learning from the original set of attributes. Hence, such analysis can help improve classification efficiency and scalability.

Data transformation and reduction:

The data may be transformed by normalization, particularly when neural networks or methods involving distance measurements are used in the learning step.

Normalization involves scaling all values for a given attribute so that they fall within a small specified range, such as -1.0 to 1.0 or 0.0 to 1.0 In methods that use distance measurements, for example, this would prevent attributes with initially large ranges (like, say, income) from out weighing attributes with initially smaller ranges (such as binary attributes).

2. Evaluating Classification and Prediction Methods

Classification and prediction methods can be compared and evaluated according to the following criteria:

Accuracy: The accuracy of a classifier refers to the ability of a given classifier to correctly predict the class label of new or previously unseen data (i.e., tuples without class label information). Similarly, the accuracy of a predictor refers to how well a given predictor can guess the value of the predicted attribute for new or previously unseen data.

Speed: This refers to the computational costs involved in generating and using the given classifier or predictor.

Robustness: This is the ability of the classifier or predictor to make correct predictions given noisy data or data with missing values.

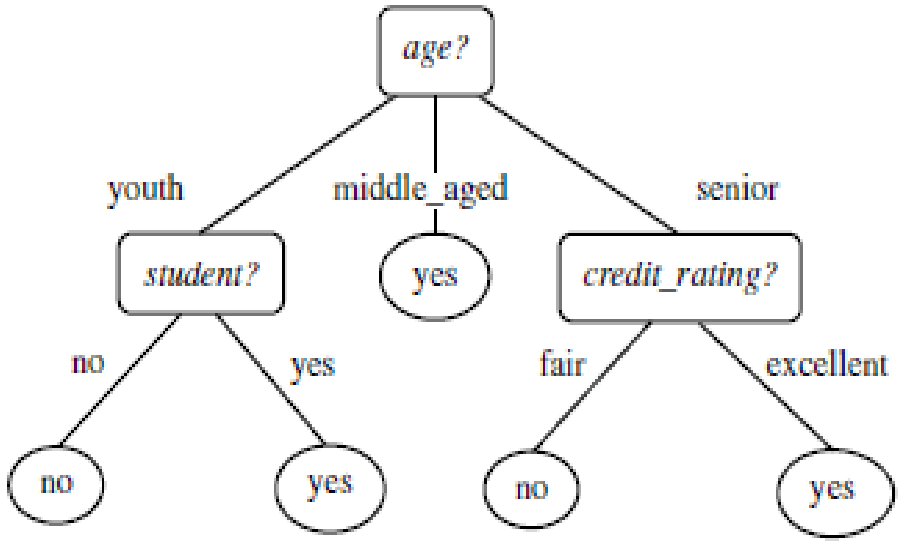
Scalability: This refers to the ability to construct the classifier or predictor efficiently given large amounts of data.

Interpretability: This refers to the level of understanding and insight that is provided by the classifier or predictor. Interpretability is subjective and therefore more difficult to assess.

CLASSIFICATION BY DECISION TREE INDUCTION

Decision tree induction is the learning of decision trees from class-labeled training tuples.

A decision tree is a flowchart-like tree structure, where each internal node (non leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (or terminal node) holds a class label. The topmost node in a tree is the root node.



1. Decision Tree Induction

During the late 1970s and early 1980s, J. Ross Quinlan, a researcher in machine learning, developed a decision tree algorithm known as ID3 (Iterative Dichotomiser).

This work expanded on earlier work on concept learning systems, described by E. B. Hunt, J. Marin, and P. T. Stone. Quinlan later presented C4.5 (a successor of ID3), which became a benchmark to which newer supervised learning algorithms are often compared.

In 1984, a group of statisticians (L. Breiman, J. Friedman, R. Olshen, and C. Stone) published the book Classification and Regression Trees (CART), which described the generation of binary decision trees. ID3 and CART were invented independently of one another at around the same time, yet follow a similar approach for learning decision trees from training tuples.

Decision tree generation consists of two phases

– Tree construction

• At start, all the training examples are at the root

• Partition examples recursively based on selected attributes

– Tree pruning

• Identify and remove branches that reflect noise or outliers

• Use of decision tree: Classifying an unknown sample Test the attribute values of the sample against the decision tree

ID3, C4.5, and CART adopt a greedy (i.e., non-backtracking) approach in which decision trees are constructed in a top-down recursive divide-and-conquer manner.

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition *D*.

Input:

- Data partition, *D*, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split point* or *splitting subset*.

Output: A decision tree.

Method:

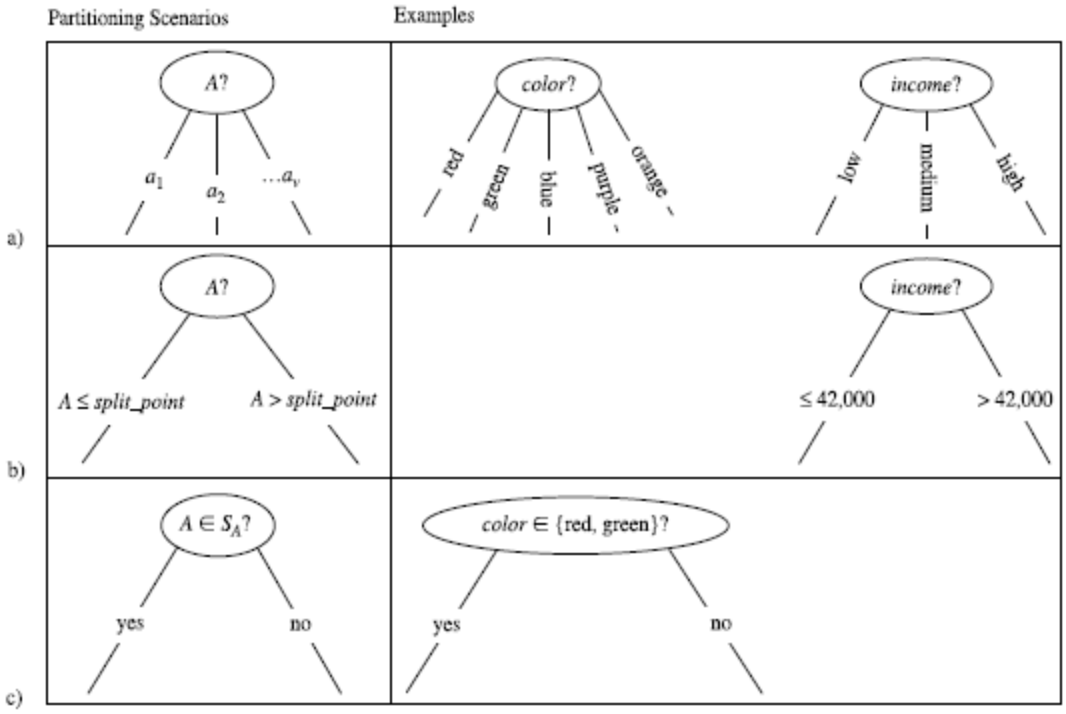
```
(1)  create a node N;
(2)  if tuples in D are all of the same class, C then
(3)      return N as a leaf node labeled with the class C;
(4)  if attribute_list is empty then
(5)      return N as a leaf node labeled with the majority class in D; // majority voting
(6)  apply Attribute_selection_method(D, attribute_list) to find the "best" splitting_criterion;
(7)  label node N with splitting_criterion;
(8)  if splitting_attribute is discrete-valued and
         multiway splits allowed then // not restricted to binary trees
(9)      attribute_list ← attribute_list − splitting_attribute; // remove splitting_attribute
(10) for each outcome j of splitting_criterion
     // partition the tuples and grow subtrees for each partition
(11)     let Dj be the set of data tuples in D satisfying outcome j; // a partition
(12)     if Dj is empty then
(13)         attach a leaf labeled with the majority class in D to node N;
(14)     else attach the node returned by Generate_decision_tree(Dj, attribute_list) to node N;
     endfor
(15) return N;
```

- Three possibilities for partitioning tuples based on the splitting criterion, shown with examples. Let A be the splitting attribute.

(a) If A is discrete-valued, then one branch is grown for each known value of A.

(b) If A is continuous-valued, then two branches are grown, corresponding to A <split point and A >= split point.

(c) If A is discrete-valued and a binary tree must be produced, then the test is of the form A belongs to S_A, where S_A is the splitting subset for A.



2. Attribute Selection Measures

Information gain

ID3 uses information gain as its attribute selection measure. Let node N represents or hold the tuples of partition D. The attribute with the highest information gain is chosen as the splitting attribute for node N.

The expected information needed to classify a tuple in D is given by

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i),$$

Where pi is the probability that an arbitrary tuple in D belongs to class C_i and is estimated by $|C_{i,D}|/|D|$.
A log function to the base 2 is used, because the information is encoded in bits.
Info (D) is just the average amount of information needed to identify the class label Of a tuple in D

How much more information would we still need (after the partitioning) in order to arrive at an exact classification? This amount is measured by

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j).$$

The term |D j|/|D| acts as the weight of the jth partition.
$Info_A(D)$ is the expected information required to classify a tuple from D based on the partitioning by A.

Information gain is defined as the difference between the original information requirement (i.e., based on just the proportion of classes) and the new requirement (i.e., obtained after partitioning on A).

$$Gain(A) = Info(D) - Info_A(D)$$

Example:

Class-labeled training tuples from the *AllElectronics* customer database.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Induction of a decision tree using information gain. Table presents a training set, D, of class-labeled tuples randomly selected from the AllElectronics customer database.

The class label attribute, buys computer, has two distinct values (namely, {yes, no}); therefore, there are two distinct classes (that is, m = 2).

Let class C1 correspond to yes and class C2 correspond to no. There are nine tuples of class yes and five tuples of class no. A (root) node N is created for the tuples in D.

To find the splitting criterion for these tuples, we must compute the information gain of each attribute

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

Next, we need to compute the expected information requirement for each attribute. Let's start with the attribute age.

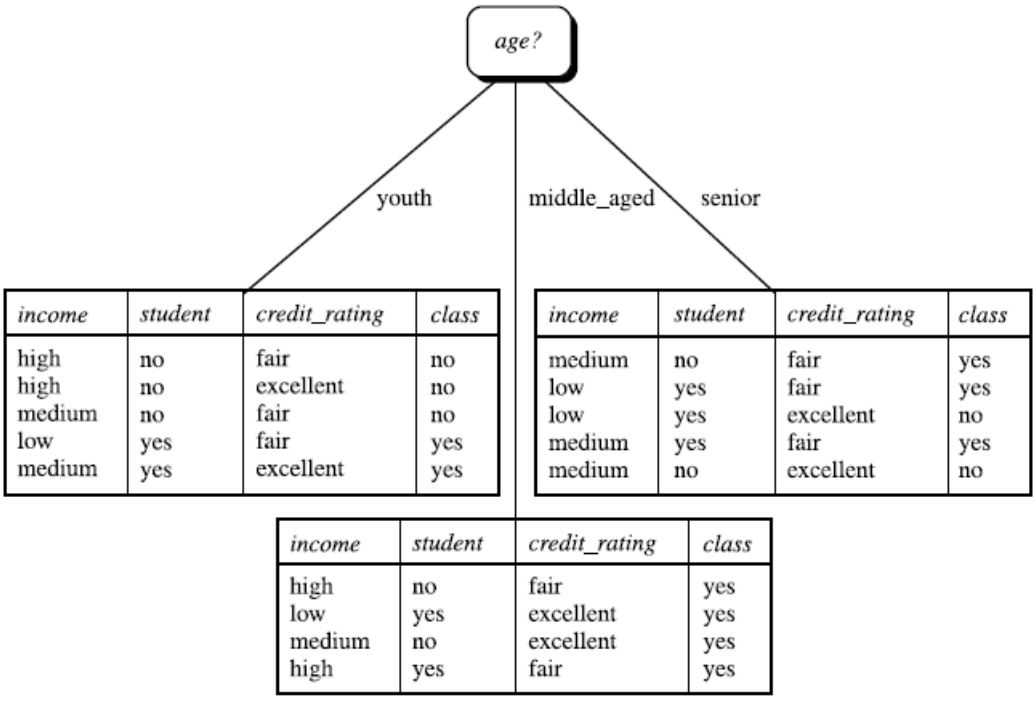
We need to look at the distribution of yes and no tuples for each category of age. For the age category youth, there are two yes tuples and three no tuples. For the category middle aged, there are four yes tuples and zero no tuples. For the category senior, there are three yes tuples and two no tuples.

$$\begin{aligned} Info_{age}(D) = &\frac{5}{14} \times \left(-\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}\right) \\ &+ \frac{4}{14} \times \left(-\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4}\right) \\ &+ \frac{5}{14} \times \left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}\right) \\ = &\ 0.694 \text{ bits.} \end{aligned}$$

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

Similarly, we can compute Gain (income) = 0.029 bits, Gain (student) = 0.151 bits, and Gain (credit rating) = 0.048 bits. Because age has the highest information gain among the attributes, it is selected as the splitting attribute.

Node N is labeled with age, and branches are grown for each of the attribute's values. The tuples are then partitioned accordingly, as shown in Figure

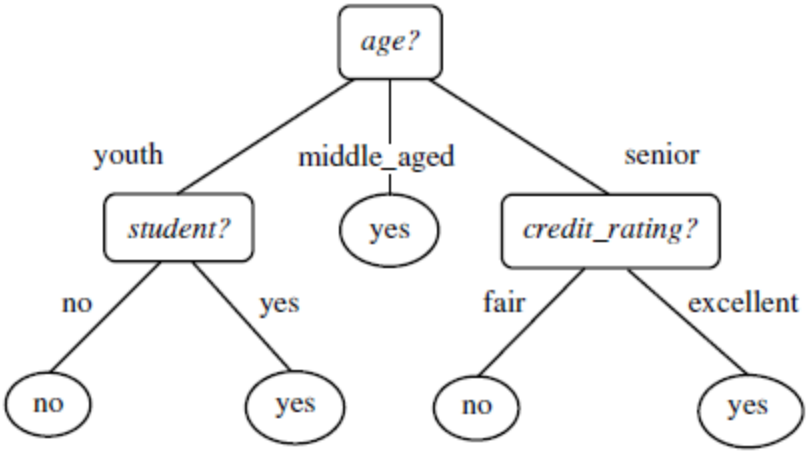


income	*student*	*credit_rating*	*class*
high	no	fair	no
high	no	excellent	no
medium	no	fair	no
low	yes	fair	yes
medium	yes	excellent	yes

income	*student*	*credit_rating*	*class*
medium	no	fair	yes
low	yes	fair	yes
low	yes	excellent	no
medium	yes	fair	yes
medium	no	excellent	no

income	*student*	*credit_rating*	*class*
high	no	fair	yes
low	yes	excellent	yes
medium	no	excellent	yes
high	yes	fair	yes

Notice that the tuples falling into the partition for age = middle aged all belong to the same class. Because they all belong to class "yes," a leaf should therefore be created at the end of this branch and labeled with "yes." The final decision tree returned by the algorithm is shown in Figure



"But how can we compute the information gain of an attribute that is continuous-valued, unlike above?" Suppose, instead, that we have an attribute A that is continuous-valued, rather than discrete-valued. (For example, suppose that instead of the discretized version of age above, we instead have the raw values for this attribute.) For such a scenario, we must determine the "best" split-point for A, where the split-point is a threshold on A.

We first sort the values of A in increasing order. Typically, the midpoint between each pair of adjacent values is considered as a possible split-point. Therefore, given v values of A, then v-1 possible splits are evaluated. For example, the midpoint between the values

a_i and a_{i+1} of A is

$(a_i+a_{i+1})/2$:

Gain ratio:

Consider an attribute that acts as a unique identifier, such as product ID. A split on product ID would result in a large number of partitions (as many as there are values), each one containing just one tuple.

C4.5, a successor of ID3, uses an extension to information gain known as gain ratio, which attempts to overcome this bias. It applies a kind of normalization to

information gain using a "split information" value defined analogously with Info(D) as

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right).$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}.$$

The attribute with the maximum gain ratio is selected as the splitting attribute

Computation of gain ratio for the attribute income. A test on income splits the data of Table into three partitions, namely low, medium, and high, containing four, six, and four tuples, respectively. To compute the gain ratio of income, we first use Equation

$$SplitInfo_A(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right).$$

$$= 0.926.$$

We have Gain (income) = 0.029. Therefore, Gain Ratio (income) =0.029/0.926 = 0.031.

Gini index

The Gini index is used in CART. Using the notation described above, the Gini index measures the impurity of D, a data partition or set of training tuples, as

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

where pi is the probability that a tuple in D belongs to class Ci and is estimated by

$|C_{i,D}|/|D|$. The sum is computed over m classes.

To determine the best binary split on A, we examine all of the possible subsets that

can be formed using known values of A. Each subset, S_A, can be considered as a binary test for attribute A of the form "A belongs S_A?"

For example, if income has three possible values, namely {low, medium, high}, then the possible subsets are {low, medium, high}, {low, medium}, {low, high}, {medium, high}, {low}, {medium}, {high}, and {}. We exclude the power set, {low, medium, high}, and the empty set from consideration since, conceptually, they do not represent a split.

When considering a binary split, we compute a weighted sum of the impurity of each resulting partition. For example, if a binary split on A partitions D into D1 and D2, the gini index of D given that partitioning is

$$Gini_A(D) = \frac{|D_1|}{|D|}Gini(D_1) + \frac{|D_2|}{|D|}Gini(D_2).$$

For each attribute, each of the possible binary splits is considered. For a discrete-valued attribute, the subset that gives the minimum gini index for that attribute is selected as its splitting subset.

The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta Gini(A) = Gini(D) - Gini_A(D).$$

Example

Let D be the training data of where there are nine tuples belonging to the class buys computer = yes and the remaining five tuples belong to the class buys computer = no. A (root) node N is created for the tuples in D.We first use Equation for Gini index to compute the impurity of D:

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

To find the splitting criterion for the tuples in D, we need to compute the gini index

for each attribute. Let's start with the attribute income and consider each of the possible splitting subsets. Consider the subset {low, medium}. This would result in 10 tuples in partition D1 satisfying the condition "income belongs {low, medium} . " The remaining four tuples of D would be assigned to partition D2. The Ginindex value computed based on this partitioning is

$$
\begin{aligned}
&Gini_{income \in \{low,medium\}}(D) \\
&\quad = \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2) \\
&\quad = \frac{10}{14}\left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \frac{4}{14}\left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) \\
&\quad = 0.443 \\
&\quad = Gini_{income \in \{high\}}(D).
\end{aligned}
$$

Similarly, the Gini index values for splits on the remaining subsets are 0.458 (for the subsets {low, high} and {medium}) and 0.450 (for the subsets {medium, high} and {low}).

Therefore, the best binary split for attribute income is on {low, medium} (or {high}) because it minimizes the Gini index. Evaluating age, we obtain {youth, senior} (or {middle aged}) as the best split for age with a Gini index of 0.375; the attributes student and credit rating are both binary, with Gini index values of 0.367 and 0.429, respectively.

The attribute age and splitting subset {youth, senior} therefore give the minimum Gini index overall, with a reduction in impurity of 0.459-0.357 = 0.102. The binary split "age belongs {youth, senior?}" results in the maximum reduction in impurity of the tuples in D and is returned as the splitting criterion.

Bayesian Classification:

Bayesian classifiers are statistical classifiers. They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class.

Bayesian classification is based on Bayes' theorem

Bayes' theorem is named after Thomas Bayes, a nonconformist English clergyman who did early work in probability and decision theory during the 18th century.

Let X be a data tuple. In Bayesian terms, X is considered "evidence." As usual, it is described by measurements made on a set of n attributes. Let H be some hypothesis, such as that the data tuple X belongs to a specified class C. For classification problems,we want to determine P(H/X), the probability that the hypothesis H holds given the "evidence" or observed data tuple X.

P(H/X) is the posterior probability, or a posteriori probability, of H conditioned

on X

Example: suppose our world of data tuples is confined to customers described by the attributes age and income, respectively, and that

X is a 35-year-old customer with an income of $40,000.

H is the hypothesis that our customer will buy a computer

P(H/X) reflects the probability that customer X will buy a computer given that we know the customer's age and income.

P(H) is the prior probability, or a priori probability, of H. For our example, this is the probability that any given customer will buy a computer, regardless of age, income

P(X/H) is the posterior probability of X conditioned on H. the probability that a customer, X, is 35 years old and earns $40,000, given that we know the customer will buy a computer.

P(X) is the prior probability of X. Using our example, it is the probability that a person from our set of customers is 35 years old and earns $40,000.

Bayes' theorem is useful in that it provides a way of calculating the posterior probability, P(H/X), from P(H), P(X/H), and P(X).

Bayes' theorem is P(H/X) =P(X/H)P(H)/P(X)

Naïve Bayesian Classification

The naïve Bayesian classifier, or simple Bayesian classifier, works as follows:

1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n-dimensional attribute vector, X = (x1, x2, : : : , xn), depicting n measurements made on the tuple from n attributes, respectively, A1, A2, : : : , An.

2. Suppose that there are m classes, C1, C2, : : : , Cm. Given a tuple, X, the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X. That is, the naïve Bayesian classifier predicts that tuple X belongs to the class Ci if and only if

P(Ci/X) > P(Cj/X) for 1 <= j <= m, j != i:

P(Ci/X) =P(X/Ci)P(Ci)/P(X)

Predicting a class label using naïve Bayesian classification. We wish to predict the class label of a tuple using naïve Bayesian classification, given the same training data as in Example for decision tree induction. The training data are in Table. The data tuples are described by the attributes age, income, student, and credit rating. The class label attribute, buys computer, has two distinct values (namely, {yes, no}).

Let C1 correspond to the class buys computer = yes and C2 correspond to buys computer = no. The tuple we wish to classify is

X = (age = youth, income = medium, student = yes, credit rating = fair)

We need to maximize P(X/Ci)P(Ci), for i = 1, 2. P(Ci), the prior probability of each

class, can be computed based on the training tuples:

P(buys computer = yes) = 9/14 = 0.643

P(buys computer = no) = 5/14 = 0.357

To compute P(X/Ci), for i = 1, 2, we compute the following conditional probabilities:

P(age = youth / buys computer = yes) = 2/9 = 0.222

P(age = youth /buys computer = no) = 3/5 = 0.600

P(income = medium / buys computer = yes) = 4/9 = 0.444

P(income = medium / buys computer = no) = 2/5 = 0.400

P(student = yes / buys computer = yes) = 6/9 = 0.667

P(student = yes / buys computer = no) = 1/5 = 0.200

P(credit rating = fair / buys computer = yes) = 6/9 = 0.667

P(credit rating = fair / buys computer = no) = 2/5 = 0.400

Using the above probabilities, we obtain

P(X/buys computer = yes) = P(age = youth /buys computer = yes) *

P(income = medium / buys computer = yes) *

P(student = yes / buys computer = yes) *

P(credit rating = fair / buys computer = yes)

= 0.222*0.444*0.667*0.667 = 0.044.

Similarly,

P(X/buys computer = no) = 0.600*0.400*0.200*0.400 = 0:019.

To find the class, Ci, that maximizes P(X/Ci)P(Ci), we compute

P(X/buys computer = yes)P(buys computer = yes) = 0.044*0.643 = 0.028

P(X/buys computer = no)P(buys computer = no) = 0.019*0.357 = 0:007

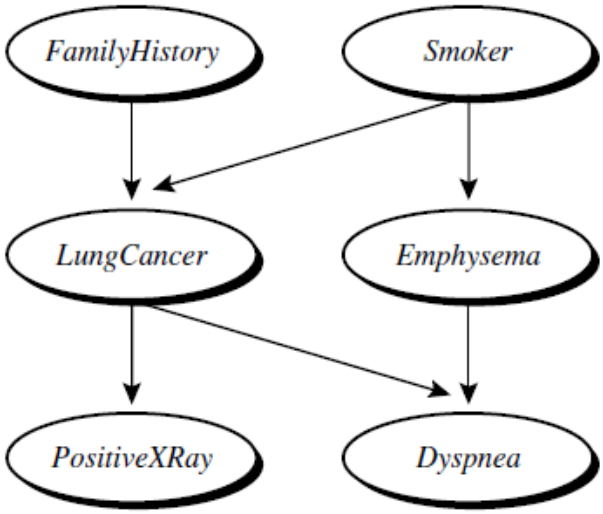
Therefore, the naïve Bayesian classifier predicts buys computer = yes for tuple X.

Bayesian Belief Networks

The naïve Bayesian classifier makes the assumption of class conditional independence however, dependencies can exist between variables. Bayesian belief networks specify joint conditional probability distributions

A belief network is defined by two components—a directed acyclic graph and a set of conditional probability table, each node in the directed acyclic graph represents a random variable. The variables may be discrete or continuous-valued. Each arc represents a probabilistic dependence. If an arc is drawn from a node Y to a node Z, then Y is a parent or immediate predecessor of Z and Z is a descendant of Y. Each variable is conditionally independent of its non-descendants in the graph, given its parents.

(a)



(b)

	FH, S	*FH, ~S*	*~FH, S*	*~FH, ~S*
LC	0.8	0.5	0.7	0.1
~LC	0.2	0.5	0.3	0.9

A simple Bayesian belief network: (a) A proposed causal model, represented by a directed acyclic graph. (b) The conditional probability table for the values of the variable *LungCancer (LC)* showing each possible combination of the values of its parent nodes, *FamilyHistory (FH)* and *Smoker (S)*.

A belief network has one conditional probability table (CPT) for each variable. The CPT for a variable Y specifies the conditional distribution P(Y/Parents(Y)), where Parents(Y) are the parents of Y. Figure shows a CPT for the variable Lung Cancer.

The conditional probability for each known value of Lung Cancer is given for each possible combination of values of its parents. For instance, from the upper leftmost and bottom rightmost entries, respectively, we see that

P(LungCancer = yes / FamilyHistory = yes, Smoker = yes) = 0.8

P(LungCancer = no / FamilyHistory = no, Smoker = no) = 0.9

Classification by Back propagation:

Back propagation is a neural network learning algorithm. A neural network is a set of connected input/output units in which each connection has a weight associated with it.

During the learning phase, the network learns by adjusting the weights so as to be able to predict the correct class label of the input tuples

Advantages of neural networks:

• High tolerance of noisy data as well as their ability to classify patterns on which they have not been trained. With little knowledge of the relationships between attributes and classes, it can be used.

• They are well suited for continuous-valued inputs and outputs, unlike most decision tree algorithms.

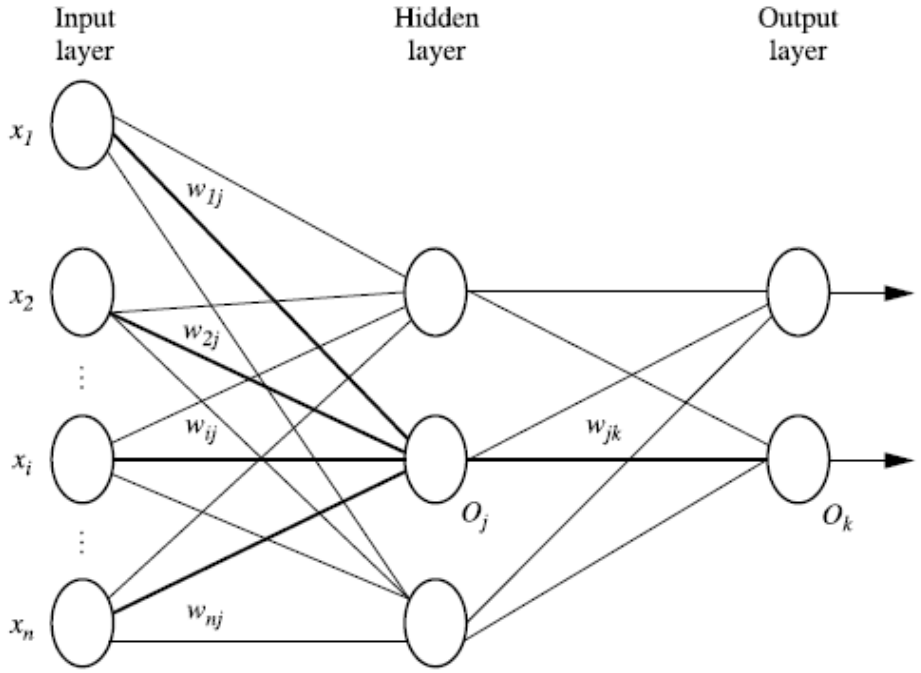
• They have been successful on a wide array of real-world data, including handwritten character recognition, pathology and laboratory medicine, and training a computer to pronounce English text.

• Neural network algorithms are inherently parallel; parallelization techniques can be used to speed up the computation process.

There are many different kinds of neural networks and neural network algorithms.The most popular neural network algorithm is backpropagation

A Multilayer Feed-Forward Neural Network

The backpropagation algorithm performs learning on a multilayer feed-forward neural network. It iteratively learns a set of weights for prediction of the class label of tuples. A multilayer feed-forward neural network consists of an input layer, one or more hidden layers, and an output layer. An example of a multilayer feed-forward network is shown in Figure



Each layer is made up of units. The inputs to the network correspond to the attributes measured for each training tuple. The inputs are fed simultaneously into the units making up the input layer. These inputs pass through the input layer and are then weighted and fed simultaneously to a second layer of "neuronlike" units, known as a hidden layer.

The outputs of the hidden layer units can be input to another hidden layer, and so on. The number of hidden layers is arbitrary, although in practice, usually only one is used. The weighted outputs of the last hidden layer are input to units making up the output layer, which emits the network's prediction for given tuples.

The units in the input layer are called input units.

The units in the hidden layers and output layer are sometimes referred to as neurodes, due to their symbolic biological basis, or as output units. The multilayer neural network shown in Figure has two layers of output units. Therefore, we say that it is a two-layer neural network. Similarly, a network containing two hidden layers is called a three-layer neural network, and so on. It is a feed-forward network since none of the weights cycles back to an input unit or to a previous layer's output unit. It is fully connected in that each unit provides input to each unit in the next forward layer.

Each output unit takes, as input, a weighted sum of the outputs from units in the previous layer. It applies a nonlinear (activation) function to the weighted input. Multilayer feed-forward neural networks are able to model the class prediction as a nonlinear combination of the inputs. From a statistical point of view, they perform nonlinear regression. Multilayer feed-forward networks, given enough hidden units and enough training samples, can closely approximate any function.

Back Propagation

Backpropagation learns by iteratively processing a data set of training tuples, comparing the network's prediction for each tuple with the actual known target value. The target value may be the known class label of the training tuple (for classification problems) or a continuous value (for numeric prediction).

The steps involved are expressed in terms of inputs, outputs, and errors, and may seem awkward if this is your first look at neural network learning. However, once you become familiar with the process, you will see that each step is inherently simple. The steps are described next. Initialize the weights: The weights in the network are initialized to small random numbers (e.g., ranging from −1.0 to 1.0, or −0.5 to 0.5). Each unit has a bias associated with it, as explained later. The biases are similarly initialized to small random numbers. Each training tuple, X, is processed by the following steps. Propagate the inputs forward: First, the training tuple is fed to the network's input layer. The inputs pass through the input units, unchanged. That is, for an input unit, j, its output, Oj , is equal to its input value, Ij . Next, the net input and output of each unit in the hidden and output layers are computed. The net input to a unit in the hidden or output layers is computed as a linear combination of its inputs.

Algorithm: Backpropagation. Neural network learning for classification or prediction, using the backpropagation algorithm.

Input:

- D, a data set consisting of the training tuples and their associated target values;
- l, the learning rate;
- *network*, a multilayer feed-forward network.

Output: A trained neural network.

Method:

(1) Initialize all weights and biases in *network*;
(2) **while** terminating condition is not satisfied {
(3) **for** each training tuple $\boldsymbol{X}$ in D {
(4) // Propagate the inputs forward:
(5) **for** each input layer unit j {
(6) $O_j = I_j$; // output of an input unit is its actual input value
(7) **for** each hidden or output layer unit j {
(8) $I_j = \sum_i w_{ij} O_i + \theta_j$; //compute the net input of unit j with respect to the previous layer, i
(9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j
(10) // Backpropagate the errors:
(11) **for** each unit j in the output layer
(12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
(13) **for** each unit j in the hidden layers, from the last to the first hidden layer
(14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
(15) **for** each weight w_{ij} in *network* {
(16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
(17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
(18) **for** each bias θ_j in *network* {
(19) $\Delta \theta_j = (l) Err_j$; // bias increment
(20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
(21) } }

Initialize the weights:

The weights in the network are initialized to small random numbers (e.g., ranging from −1.0 to 1.0, or −0.5 to 0.5). Each unit has a bias associated with it, as explained later.

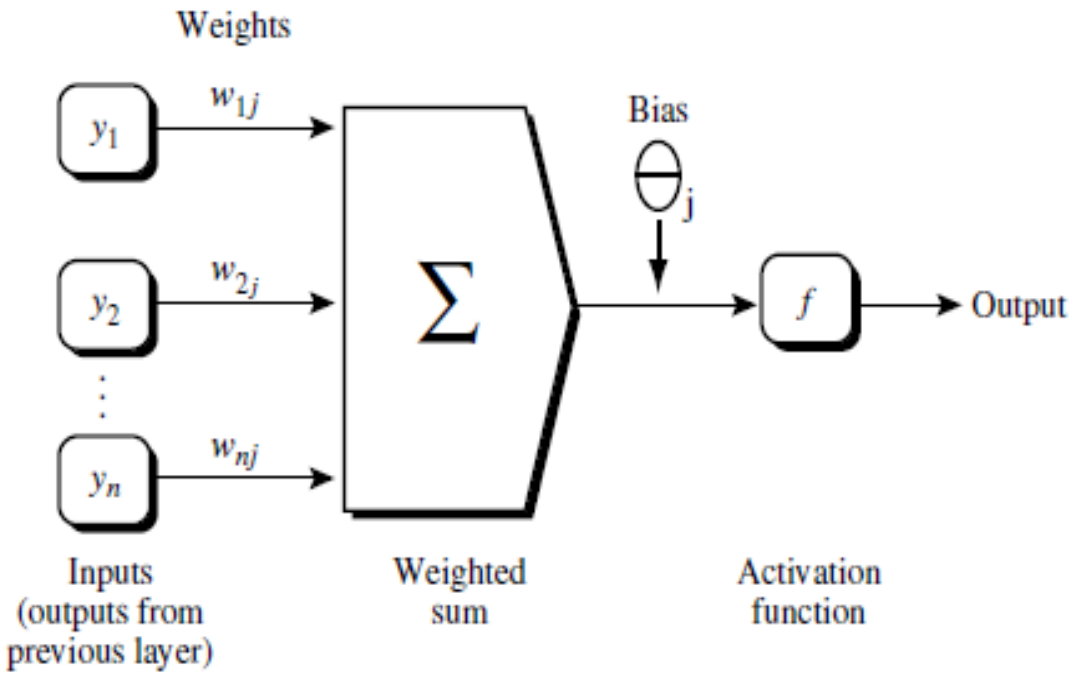
The biases are similarly initialized to small random numbers. Each training tuple, X, is processed by the following steps.

Propagate the inputs forward:

First, the training tuple is fed to the network's input layer. The inputs pass through the input units, unchanged. That is, for an input unit, j, its output, Oj , is equal to its input value, Ij .

Next, the net input and output of each unit in the hidden and output layers are computed. The net input to a unit in the hidden or output layers is computed as a linear combination of its inputs.

The diagram is shown below



Each such unit has a number of inputs to it that are, in fact, the outputs of the units connected to it in the previous layer. Each connection has a weight.

To compute the net input to the unit, each input connected to the unit is multiplied by its corresponding weight, and this is summed.

Given a unit j in a hidden or output layer, the net input, Ij, to unit j is

$$I_j = \sum_i w_{ij} O_i + \theta_j,$$

where wi j is the weight of the connection from unit i in the previous layer to unit j; Oi is the output of unit i from the previous layer; and qj is the bias of the unit. The bias acts as a threshold in that it serves to vary the activity of the unit.

Each unit in the hidden and output layers takes its net input and then applies an activation function to it, as illustrated above. The function symbolizes the activation of the neuron represented by the unit. The logistic, or sigmoid, function is used. Given the net input Ij to unit j, then Oj, the output of unit j, is computed as

$$O_j = \frac{1}{1 + e^{-I_j}}.$$

Backpropagate the error: The error is propagated backward by updating the weights and biases to reflect the error of the network's prediction. For a unit j in the output layer, the error

Err j is computed by

$$Err_j = O_j(1 - O_j)(T_j - O_j),$$

where Oj is the actual output of unit j, and Tj is the known target value of the given training tuple. Note that Oj(1-Oj) is the derivative of the logistic function.

To compute the error of a hidden layer unit j, the weighted sums of the errors of the units connected to unit j in the next layer are considered. The error of a hidden layer unit j is

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk},$$

where wjk is the weight of the connection from unit j to a unit k in the next higher layer, and Errk is the error of unit k.

The weights and biases are updated to reflect the propagated errors. Weights are updated by the following equations, where D wi j is the change in weight wi j

$$\Delta w_{ij} = (l) Err_j O_i$$
$$w_{ij} = w_{ij} + \Delta w_{ij}$$

The variable l is the learning rate, a constant typically having a value between 0.0 and 1.0. Backpropagation learns using a method of gradient descent to search for a set of weights that fits the training data so as to minimize the mean squared distance between the network's class prediction and the known target value of the tuples. The learning rate helps avoid getting stuck at a local minimum in decision space (i.e., where the weights appear to converge, but are not the optimum solution) and encourages finding the global minimum. If the learning rate is too small, then learning will occur at a very slow pace. If the learning rate is too large, then oscillation between inadequate solutions may occur. A rule of thumb is to set the learning rate to 1/t, where t is the number of iterations through the training set so far.

Biases are updated by the following equations below, where D q j is the change in

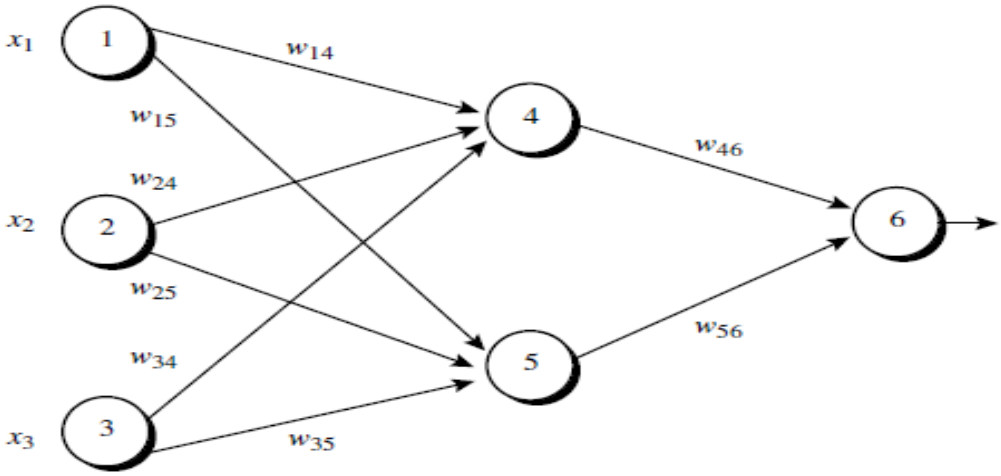
bias q j:

$$\Delta \theta_j = (l) Err_j$$

$$\theta_j = \theta_j + \Delta \theta_j$$

Sample calculations for learning by the backpropagation algorithm.

Let the learning rate be 0.9. The initial weight and bias values of the network are given below, along with the first training tuple, X = (1, 0, 1), whose class label is 1.



An example of a multilayer feed-forward neural network.

Initial input, weight, and bias values.

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	−0.3	0.4	0.1	−0.5	0.2	−0.3	−0.2	−0.4	0.2	0.1

The net input and output calculations.

Unit j	*Net input, I_j*	*Output, O_j*
4	$0.2+0-0.5-0.4=-0.7$	$1/(1+e^{0.7})=0.332$
5	$-0.3+0+0.2+0.2=0.1$	$1/(1+e^{-0.1})=0.525$
6	$(-0.3)(0.332)-(0.2)(0.525)+0.1=-0.105$	$1/(1+e^{0.105})=0.474$

Calculation of the error at each node.

Unit j	*Err_j*
6	$(0.474)(1-0.474)(1-0.474)=0.1311$
5	$(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065$
4	$(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087$

Calculations for weight and bias updating.

Weight or bias	*New value*
w_{46}	$-0.3+(0.9)(0.1311)(0.332)=-0.261$
w_{56}	$-0.2+(0.9)(0.1311)(0.525)=-0.138$
w_{14}	$0.2+(0.9)(-0.0087)(1)=0.192$
w_{15}	$-0.3+(0.9)(-0.0065)(1)=-0.306$
w_{24}	$0.4+(0.9)(-0.0087)(0)=0.4$
w_{25}	$0.1+(0.9)(-0.0065)(0)=0.1$
w_{34}	$-0.5+(0.9)(-0.0087)(1)=-0.508$
w_{35}	$0.2+(0.9)(-0.0065)(1)=0.194$
θ_6	$0.1+(0.9)(0.1311)=0.218$
θ_5	$0.2+(0.9)(-0.0065)=0.194$
θ_4	$-0.4+(0.9)(-0.0087)=-0.408$

Prediction:

"What if we would like to predict a continuous value, rather than a categorical label?" Numeric prediction is the task of predicting continuous (or ordered) values for given input. For example, we may wish to predict the salary of college graduates with 10 years of work experience. The most widely used approach for numeric prediction (hereafter referred to as prediction) is regression.

Regression analysis can be used to model the relationship between one or more independent or predictor variables and a dependent or response variable.

Linear Regression:

Straight-line regression analysis involves a response variable, y, and a single predictor variable, x. It is the simplest form of regression, and models y as a linear function of x.

That is,

$$y = b+wx;$$

where the variance of y is assumed to be constant, and b and w are regression coefficients specifying the Y-intercept and slope of the line, respectively. The regression coefficients, w and b, can also be thought of as weights, so that we can equivalently write,

$$y = w0+w1x$$

These coefficients can be solved for by the method of least squares, which estimates the best-fitting straight line as the one that minimizes the error between the actual data and the estimate of the line.

Let D be a training set consisting of values of predictor variable, x, for some population and their associated values for response variable, y. The training set contains |D| data points of the form(x1, y1), (x2, y2), : : : , ($x_{|D|}$, $y_{|D|}$).The regression coefficients can be estimated using this method with the following equations:

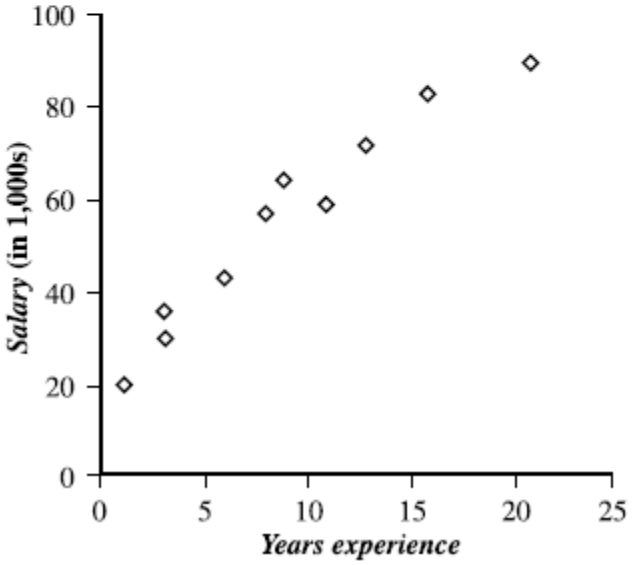
$$w_1 = \frac{\sum_{i=1}^{|D|}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|}(x_i - \bar{x})^2}$$

$$w_0 = \bar{y} - w_1\bar{x}$$

where $\bar{x}$ is the mean value of $x_1, x_2, \ldots, x_{|D|}$, and $\bar{y}$ is the mean value of $y_1, y_2, \ldots, y_{|D|}$. The coefficients w_0 and w_1 often provide good approximations to otherwise complicated regression equations.

Salary data.

x years experience	y salary (in \$1000s)
3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83



We model the relationship that salary may be related to the number of years of work experience with the equation $y = w_0 + w_1 x$.

Given the above data, we compute $\bar{x} = 9.1$ and $\bar{y} = 55.4$. Substituting these values into Equations (6.50) and (6.51), we get

$$w_1 = \frac{(3-9.1)(30-55.4)+(8-9.1)(57-55.4)+\cdots+(16-9.1)(83-55.4)}{(3-9.1)^2+(8-9.1)^2+\cdots+(16-9.1)^2} = 3.5$$

$$w_0 = 55.4-(3.5)(9.1) = 23.6$$

Thus, the equation of the least squares line is estimated by $y = 23.6 + 3.5x$. Using this equation, we can predict that the salary of a college graduate with, say, 10 years of experience is \$58,600. ■

Accuracy Classifier:

Suppose you used data from previous sales to train a classifier to predict customer purchasing behavior. You would like an estimate of how accurately the classifier can predict the purchasing behavior of future customers, that is, future customer data on which the classifier has not been trained. You may even have tried different methods to build more than one classifier (or predictor) and now wish to compare their accuracy. But what is accuracy? How can we estimate it? Are there strategies for increasing the accuracy of a learned model?

	Predicted Yes	Predicted No
Actual Yes	TP	FN
Actual No	FP	TN

Confusion Matrix

Accuracy= (TN+TP)/(TN+FP+FN+TP)

Precesion=TP/Pred Yes

Sensitivty=TP/Actual Yes

Specificity=TN/Actual No

Predictor Error Measures

"*How can we measure predictor accuracy?*" Let D^T be a test set of the form $(\boldsymbol{X_1}, y_1)$, $(\boldsymbol{X_2}, y_2)$, ... , $(\boldsymbol{X_d}, y_d)$, where the $\boldsymbol{X_i}$ are the n-dimensional test tuples with associated known values, y_i, for a response variable, y, and d is the number of tuples in D^T. Since predictors return a continuous value rather than a categorical label, it is difficult to say *exactly* whether the predicted value, y_i', for $\boldsymbol{X_i}$ is correct. Instead of focusing on whether y_i' is an "exact" match with y_i, we instead look at how far off the predicted value is from the actual known value. Loss functions measure the error between y_i and the predicted value, y_i'. The most common loss functions are:

$$\text{Absolute error}: \quad |y_i - y_i'|$$

$$\text{Squared error}: \quad (y_i - y_i')^2$$

$$\textbf{Mean absolute error}: \quad \frac{\sum_{i=1}^{d} |y_i - y_i'|}{d}$$

$$\textbf{Mean squared error}: \quad \frac{\sum_{i=1}^{d} (y_i - y_i')^2}{d}$$

The mean squared error exaggerates the presence of outliers, while the mean absolute error does not. If we were to take the square root of the mean squared error, the resulting error measure is called the **root mean squared error**. This is useful in that it allows the error measured to be of the same magnitude as the quantity being predicted.

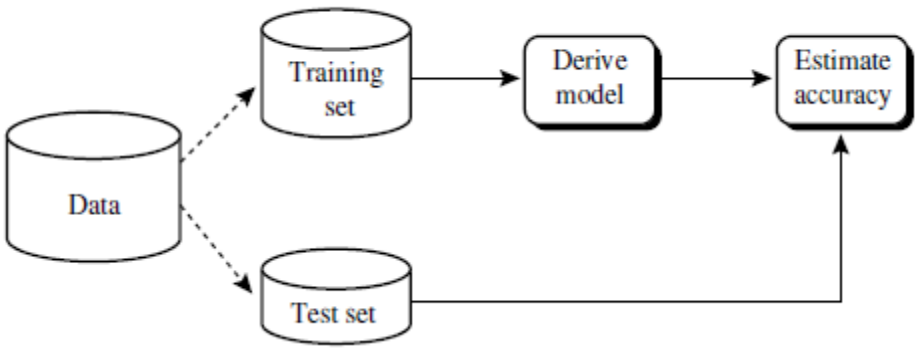
Sometimes, we may want the error to be relative to what it would have been if we had just predicted $\bar{y}$, the mean value for y from the training data, D. That is, we can normalize the total loss by dividing by the total loss incurred from always predicting the mean. Relative measures of error include:

$$\textbf{Relative absolute error}: \quad \frac{\sum_{i=1}^{d} |y_i - y_i'|}{\sum_{i=1}^{d} |y_i - \bar{y}|}$$

$$\textbf{Relative squared error}: \quad \frac{\sum_{i=1}^{d} (y_i - y_i')^2}{\sum_{i=1}^{d} (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean value of the y_i's of the training data, that is $\bar{y} = \frac{\sum_{i=1}^{t} y_i}{d}$.

Evaluating the Accuracy of a Classifier or Predictor



Holdout Method

The holdout method is what we have alluded to so far in our discussions about accuracy. In this method, the given data are randomly partitioned into two independent sets, a training set and a test set. Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set. The training set is used to derive the model, whose accuracy is estimated with the test set. The estimate is pessimistic because only a portion of the initial data is used to derive the model.

Random subsampling is a variation of the holdout method in which the holdout method is repeated k times. The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration. (For prediction, we can take the average of the predictor error rates.)

Cross-validation

In k-fold cross-validation, the initial data are randomly partitioned into k mutually exclusive subsets or "folds," D1, D2, : : : , Dk, each of approximately equal size. Training and testing is performed k times. In iteration i, partition Di is reserved as the test set, and the remaining partitions are collectively used to train the model. That is, in the first iteration, subsets D2, : : : , Dk collectively serve as the training set in order to obtain a first model, which is tested on D1; the second iteration is trained on subsets D1, D3, : : : , Dk and tested on D2; and so on. Unlike the holdout and random subsampling methods above, here, each sample is used the same number of times for training and once for testing. For classification, the accuracy estimate is the overall number of correct classifications from the k iterations, divided by the total number of tuples in the initial data. For prediction, the error estimate can be computed as the total loss from the k iterations, divided by the total number of initial tuples.

Bootstrap

Unlike the accuracy estimation methods mentioned above, the bootstrap method samples the given training tuples uniformly with replacement. That is, each time a tuple is selected, it is equally likely to be selected again and readded to the training set. For instance, imagine a machine that randomly selects tuples for our training

set. In sampling with replacement, the machine is allowed to select the same tuple more than once.

There are several bootstrap methods. A commonly used one is the .632 bootstrap, which works as follows. Suppose we are given a data set of d tuples. The data set is sampled d times, with replacement, resulting in a bootstrap sample or training set of d samples. It is very likely that some of the original data tuples will occur more than once in this sample. The data tuples that did not make it into the training set end up forming the test set.

Other Classification methods:

- Genetic Algorithm
- Rough Set Algorithm
- Fuzzy Logic Algorithm